Engineering Notebook Submission - Savanna Coffel, Ian Moore

Here are some of the initial sketches from Savanna's notebook that outline the structure of the .proto file and the RPC specs:

Design Exercise 2 - 02/20/25 - Initial Specs and Plans — Engineering Notebook: Savanna Coffel

Goal: replace using custom socket protocol/JSON with gRPC

- does using gRPC make the app easier? ⇒ I expect the GUI to be unchanged, so no
- how does it change the size of the data passed? ⇒ Should shrink the data b/c we send it serialized
- how does it change client/server structure? ⇒ will need a .proto file and references to it in Server & Client

RPC Specs/Gathering Info:

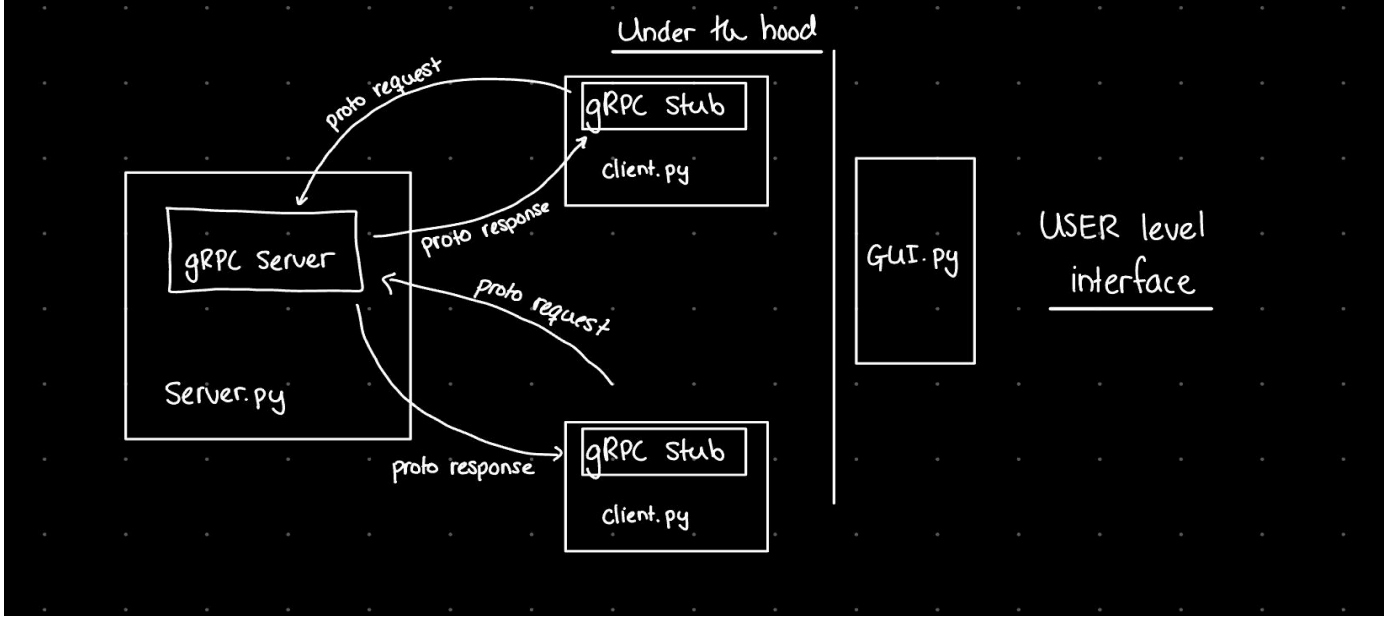
→ IDL: protocol buffers ⇒ protobuf smaller/more compact than text-based formatting (JSON & our protocol were text-based) ⇒ protobuf is binary

→ transport protocol: HTTP/2

→ define data in .proto file

↳ (holds service definitions & messages)

↳ compiled with protoc (protocol buffers compiler) & gRPC plugin (so we can use python)



Structuring the .proto file

universal whether we use python or not

Syntax defs `syntax = "proto3";` //tells compiler which proto syntax I'm using

import client and server here?

declaring packages `package app;` //when I can import client & server functions

imports

message definitions

need to talk to Ian about how we format this; want to preserve preexisting message structure in dict that we have already

```
message RequesttoServer {
    string name = 1;
    int32 age = 2;
    repeated string hobbies = 3;
}
```

What are these?

enumerations (enums)

Service methods (define how servers respond to client requests)

command line probably?

(compile with `python -m grpc_tools.protoc \`
`:`
`paths`)

→ we should talk about service methods - I want this to be as similar as possible to our previous implementation

→ I do not want to change any of our data handling/account management functions. We can get away with leaving them as is because they're independent of the wire protocol

Our postmortem after the gRPC refactor:

TLDR: We are big fans of the gRPC refactor overall!

Does the tool make the application easier or more difficult?
The client and server are more compact now in terms of the size of the codebase. This makes the code much more manageable and easier to document, maintain, and understand. The functions are split up into more digestible bits, and everything is modularized. We find that gRPC made our code much simpler and more elegant.

How does it change the structure of the client? The server?
We still have that the server is constantly listening for client input, just not on a socket - that sort of functionality is now handled on the backend by gRPC. We don't have to have the same loops - which we found obnoxious and hard to control - to listen on the socket and manage communication. For both the client and the server functions, we only need to make a single call to the client and server stubs, respectively, to handle the message passing between the two. This means we can design functions in both scripts that are much simpler and don't necessitate constant communication on the wire.

We really like that everything is now managed on the .proto file. This makes the design of the application and keeping track of the communication between server and client much easier to manage, understand, and update.

We also like that this is platform agnostic in terms of the communication occurring via the .proto-defined protocol. This means that in terms of future extensibility, the GRPC implementation could potentially allow for any other service in another language to be linked into our distributed system.

Really gRPC is doing the heavy lifting here in terms of generating the compiler code that abstracts away a lot of the communication challenges. This results in a much nicer, cleaner, and shorter codebase.

From the user's perspective, the GUI still functions exactly the same. We've tried to keep this as simple as possible for the user, and not having to refactor or GUI code made switching over to the new protocol extremely straightforward.

How does this change the testing of the application?
We created analogous tests for the gRPC implementation that we had for the JSON and custom wire protocol encoding formulations (see wireprotocolsizetesting.py).

There are some critical points to note:

- We are excluding transport level overhead, whether it is TCP with the JSON/custom wire protocol or the gRPC communication overhead.

- That is to say, we only measure raw bytes, whether encoded in JSON, via the custom wire protocol, or using the gRPC protobuf protocol.

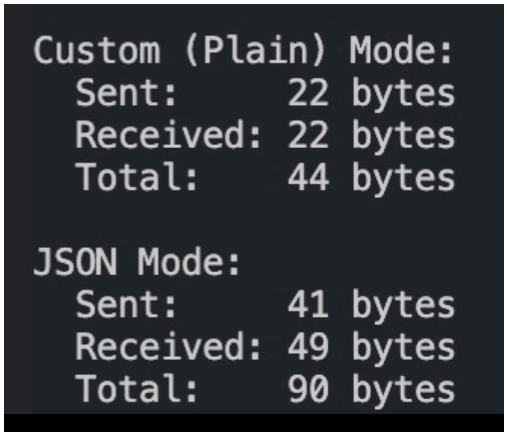
- Our new formulation, instead of imitating a socket, imitates a gRPC stub to allow for testing. (Basically, we simulate a server.)

- However, the test messages passed between mock stubs (one out, one in) are the same type and content as in our previous tests, so it is an apples-to-apples comparison.

What does it do to the size of the data passed?
Here are the results:

```
Protobuf Mode:
  Sent:     34 bytes
  Received: 22 bytes
  Total:    56 bytes
```

Here is a comparison to the implementations we had before:



We see that in the new implementation using gRPC, we are substantially more efficient with the protobuf implementation than with JSON mode, but our custom mode was still more efficient.

The reasons for this should be clear considering the inherent inefficiencies in encoding content in JSON (the data structure created is just inherently inefficient in terms of bytes because of the keyword encodings), as well as by looking at the proto file that defines the different calls and seeing the additional params defined in the gRPC communication that do not occur in the custom wire protocol.

The gRPC communication is very, very efficient, but because of the way we defined arguments in the .proto file, we have some additional data being passed around to keep the communication consistent and the format normalized, compared to our custom protocol from before.

Our custom wire protocol on occasion omits certain fields that our gRPC formulation is passing, which makes our custom wire protocol slightly more ‘efficient’ than our gRPC formulation since less data is being passed. It is, however, less normalized.

Overall these minor inefficiencies (if we could even call them that, as the gRPC formulation is functionally much nicer than our custom wire protocol) definitely do not outweigh all the benefits we outlined above of using the gRPC implementation.